

Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2024/2025

Interação em Ambientes Virtuais

Trabalho Prático Nº2

Turma: 61D

Nome: João Ramos Número: 50730

Nome: Fábio Pestana Número: 50756

Professor: Diogo Cabral

Data: 11 de maio de 2025

Índice

Lista de Figuras 3
Lista de Tabelas 4
Introdução 5
O que é e como funciona *Machine Learning*? 6
ML-Agents 6
HelloWorld 7
FoodButton 8
PathCreator 10
Mas o que são curvas de Bézier? 10
ProceduralRoad3D 10
Ray Perception Sensor 11
Manager Scenes 13
Conclusão 15
Bibliografia 16

Lista de Figuras

Figura 1 - Diferença entre ML e programas tradicionais 6
Figura 2 - Ciclo do step de aprendizagem 7
Figura 3 - Simulação HelloWorld 8
Figura 4 - Simulação FoodButton 10
Figura 5 - Teste da ProceduralRoad3D 11
Figura 6 - Simulação RayPerceptionSensor 13
Figura 7 - Interface para escolha de uma cena 13
Figura 8 - Simulação RayPerceptionSensor apartir do ManagerScenes 14

Lista de Tabelas

Tabela 1 - Diferenças entre a utilização do rb.velocity e do rb.AddForce 9
Tabela 2 - Comparação entre configurações do agente 12

Introdução

Este trabalho tem como objetivo o desenvolvimento de um ambiente de aprendizagem utilizando o motor de jogo Unity. Os alunos tiveram como base o código desenvolvido nas aulas, com o intuito de criar agentes e treinar os mesmos. Um dos pontos principais deste trabalho é a utilização de *Machine Learning* através de algoritmos de aprendizagem por reforço para treinar os agentes de modo a atingir os melhores resultados possíveis.

O que é e como funciona *Machine Learning*?

Este projeto da unidade curricular Interação em Ambientes Virtuais utiliza *Machine Learning* que é um campo de estudo na área de inteligência artificial que tem como objetivo desenvolver algoritmos capazes de aprender através de dados e resultados obtidos de modo a criar a melhor resolução para um problema ou tarefa. O esquema abaixo demonstra a diferença entre um programa tradicional e um algoritmo de *Machine Learning*:

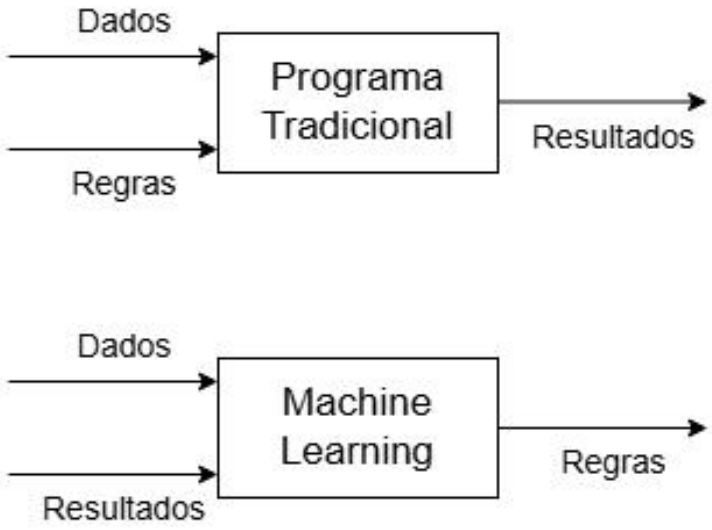


Figura 1 - Diferença entre ML e programas tradicionais

ML-Agents

Este projeto recorre ao **ML-Agents Toolkit** que é uma ferramenta desenvolvida para Unity. Esta fornece um conjunto de componentes que ajudam no treino de agentes inteligentes dentro do motor de jogo Unity, o que permite uma aprendizagem de comportamentos através de interações com o ambiente, com base em técnicas como aprendizagem por reforço.

O ML-Agents é composto por dois principais componentes: o ambiente Unity e uma API Python que permite treinar os agentes ao utilizar algoritmos como PPO (*Proximal Policy Optimization*) e SAC (*Soft Actor-Critic*). De modo a existir comunicação entre o Unity e o Python é utilizado o protocolo gRPC que permite o envio de observações, ações e *feedback* coletadas no ambiente Unity para o treinador em Python.

Para compreender melhor como os agentes evoluem durante o treino, é necessário analisar o funcionamento dos *steps* de aprendizagem que compõem o processo de interação e melhoria do agente.

Os *steps* de aprendizagem representam as unidades de tempo em que ocorrem as interações entre o agente e o ambiente. A cada *step*, o agente recebe uma observação do estado atual do ambiente, toma uma decisão adequada, executa uma ação com base na decisão escolhida e recebe uma recompensa como *feedback* do seu desempenho. Este ciclo repete-se continuamente e está ilustrado na figura abaixo:

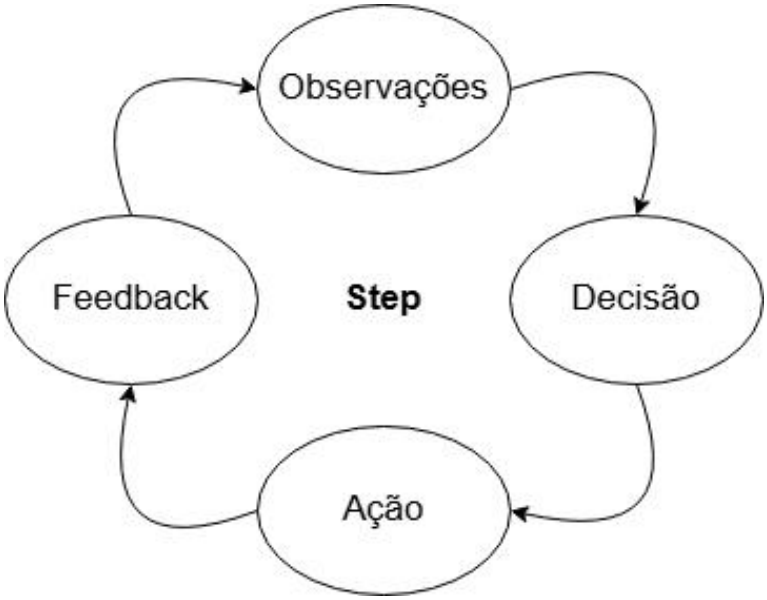


Figura 2 - Ciclo do step de aprendizagem

HelloWorld

Para testarmos esta forma de treino do agente, foi desenvolvida uma simples simulação em que um agente tem de se deslocar até tocar num *target*. Nesta simulação, o agente é treinado para se deslocar até um *target* enquanto evita tocar nas paredes. A cada *step*, o agente recebe como *input* (observações) duas informações críticas: a sua própria posição (em coordenadas X e Z) e a posição do alvo. Estas observações são essenciais para que o modelo de aprendizagem por reforço possa tomar decisões dada a informação recebida. Com base nestas informações, o agente decide em que direção se deve mover, produzindo assim duas ações contínuas correspondentes ao movimento nos eixos X e Z.

Após executar a ação, o agente pode receber diferentes tipos de *feedback*: se tocar no *target*, recebe uma recompensa positiva (+1), e se colidir com uma parede, uma penalização (-1).

Com base nas recompensas acumuladas, o algoritmo de aprendizagem por reforço ajusta os pesos da rede neural, refinando gradualmente a estratégia do agente para maximizar a recompensa total.

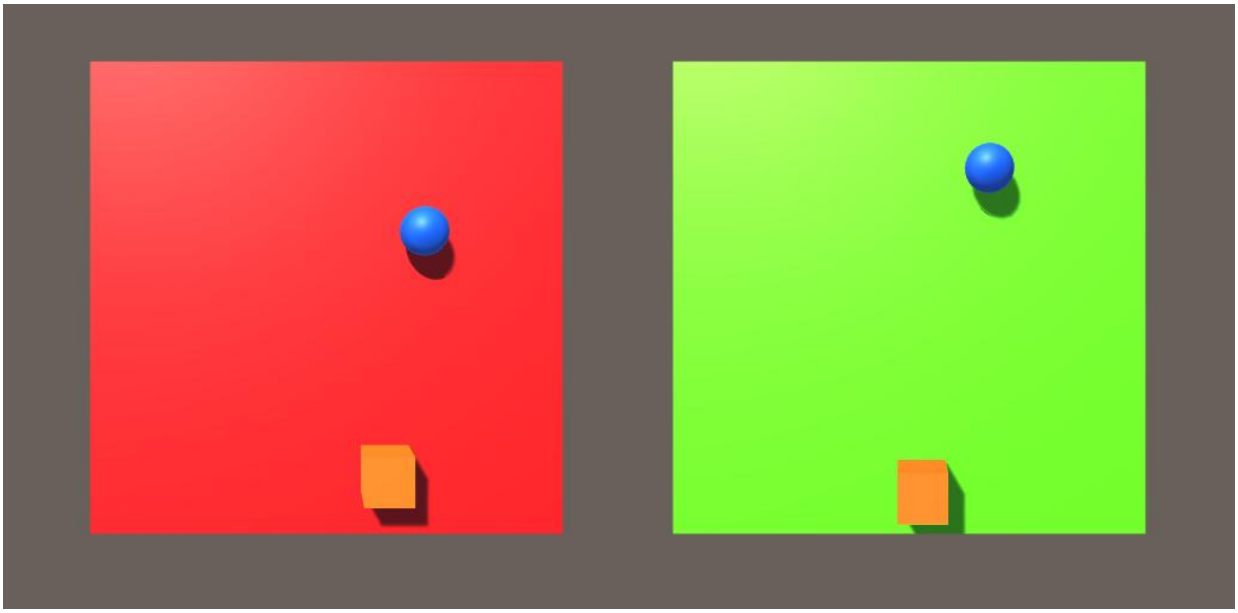


Figura 3 - Simulação HelloWorld

Na figura 3, encontra-se a simulação do agente (esfera azul), do lado esquerdo encontra-se um exemplo da simulação quando o agente colide com uma parede e do lado direito encontra-se um exemplo da simulação quando o agente atinge o seu objetivo (tocar no *target*).

FoodButton

Posteriormente, implementámos uma segunda simulação com um maior nível de complexidade. Nesta nova tarefa, o agente deve realizar uma sequência de ações para alcançar o objetivo final: primeiro, deve deslocar-se até um botão e pressioná-lo, o que faz aparecer um objeto "comida". Em seguida, o agente precisa dirigir-se até esse objeto para o consumir. Tal como na simulação anterior, o agente deve evitar colidir com as paredes, pois qualquer contacto com estas termina imediatamente o episódio. Para reforçar esse comportamento, o agente recebe uma recompensa negativa sempre que toca numa parede, sinalizando claramente que essa ação deve ser evitada. Esta cadeia de ações exige do agente não apenas uma boa perceção do ambiente, mas também a capacidade de tomar decisões que dependem de eventos anteriores, tornando o problema mais complexo.

O sistema de recompensas foi desenhado para guiar o comportamento desejado: o agente recebe uma recompensa positiva ao pressionar o botão e outra ao consumir a comida. Além disso, a cada *step* do episódio, é atribuída uma pequena penalização negativa, incentivando o agente a resolver a tarefa no menor número de passos possíveis e a evitar movimentos desnecessários.

Foram aplicadas diferentes abordagens de aprendizagem nesta simulação. Inicialmente, utilizou-se a aprendizagem por reforço convencional, mas os resultados obtidos não foram satisfatórios, visto que a recompensa média (em cada dez mil episódios) acabava por ser negativa. Como resposta, implementou-se a técnica de treino com múltiplos *environments* em paralelo (paralelismo), permitindo que o agente acumulasse mais experiências em menos tempo, acelerando o processo de aprendizagem.

Outra estratégia testada foi a aprendizagem por imitação, onde o agente tenta aprender com base em demonstrações humanas sobre como resolver a tarefa. No entanto, esta técnica não apresentou os resultados esperados neste contexto específico, o que levou a um novo ajuste: aumentou-se o número de `stacked vectors` para dois, permitindo ao agente ter acesso às observações de dois momentos consecutivos, em vez de apenas um (*frame* atual e o anterior).

Os `stacked vectors` são uma técnica que consiste em empilhar múltiplos conjuntos de observações passadas para fornecer ao agente uma noção temporal. Com esta informação adicional, o agente consegue ter uma perceção melhor da dinâmica do ambiente, o que o ajuda a tomar melhores decisões, especialmente em tarefas que exigem memória ou ações sequenciais, como é o caso desta simulação. Com esta configuração, e sem a utilização de gravações humanas (apenas utilizou-se paralelismo), o agente apresentou um desempenho significativamente superior, conseguindo resolver a tarefa de forma consistente após o treino.

Esta simulação foi realizada inicialmente utilizando `rb.velocity = force * strength` para movimentar o agente, pois esta abordagem oferece um controle direto e imediato sobre a velocidade do objeto, simplificando o aprendizado nas fases iniciais. Posteriormente, para tornar o problema mais complexo e fisicamente realista, optou-se por utilizar `rb.AddForce(force * strength)`, que aplica forças físicas cumulativas ao Rigidbody, resultando em movimentos mais orgânicos e sujeitos às leis da física. Na seguinte tabela, encontram-se os resultados finais do treino dos agentes (resultados retirados do **TensorBoard**, gráfico das recompensas):

Método	Smoothed Reward	Raw Reward (Value)
`rb.velocity`	1.5547	1.5737
`rb.AddForce`	1.1418	1.1080

Tabela 1 - Diferenças entre a utilização do rb.velocity e do rb.AddForce

Tal como era esperado, a simulação com o agente sem a aplicação de forças apresenta melhores resultados (média das recompensas mais alta), isto indica que o agente conseguiu aprender melhor a sua tarefa (pressionar o botão e obter a comida), enquanto que a simulação com física, devido ao movimento ser baseado em aplicação de forças, leva a que a esfera tenha mais insucesso no treino, pois nesta simulação utilizamos apenas ações discretas, logo não é possível o agente aplicar diferentes valores de força segundo a sua distância às paredes. O grupo obtou por utilizar a versão com `rb.AddForce`, pois consideramos mais interessante e mais realista o movimento da esfera nesta simulação.

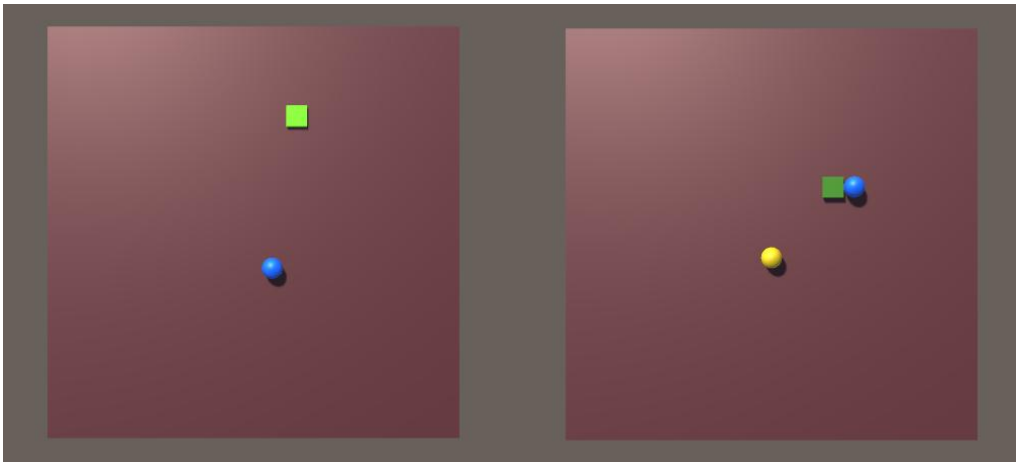


Figura 4 - Simulação FoodButton

Na figura 4, encontra-se a simulação do agente (esfera azul), do lado esquerdo encontra-se o início da simulação (objetivo do agente é pressionar o botão) e do lado direito encontra-se um exemplo da simulação quando o agente pressiona o botão e tem como objetivo tocar na comida (esfera amarela).

PathCreator

O `PathCreator` é um *asset* do Unity que será utilizado posteriormente, que permite criar e manipular caminhos de forma visual dentro da cena. Com esta ferramenta, é possível desenhar percursos usando curvas de Bézier, ajustando pontos de controlo diretamente no editor. Além disso, o `PathCreator` fornece métodos para obter posições, rotações e direções ao longo do caminho com base na distância percorrida, o que facilita o movimento suave de objetos ao longo do trajeto.

Mas o que são curvas de Bézier?

As curvas de Bézier são curvas polinomiais utilizadas em computação gráfica e modelação para representar trajetórias suaves. São definidas por um conjunto de pontos de controlo, onde o primeiro e o último ponto determinam o início e o fim da curva, e os pontos intermédios influenciam a curvatura sem necessariamente estar na linha da curva. A forma mais comum em aplicações como o Unity é a curva de Bézier cúbica, que utiliza quatro pontos: dois pontos de ancoragem (inicial e final) e dois pontos de controlo que definem a direção e intensidade da curvatura. Estas curvas são utilizadas no `PathCreator`, visto que permitem movimentos naturais e contínuos ao longo dos caminhos.

ProceduralRoad3D

Nesta simulação, o objetivo foi desenvolver um sistema que gera estradas de forma procedimental, adaptando-se dinamicamente ao terreno e criando um percurso contínuo para um veículo, utilizando o *asset* `PathCreator` para

Unity. A implementação utiliza algoritmos de *Perlin Noise* (já utilizado no trabalho prático Nº1) com geração de malha (`mesh`) para criar trajetos naturais e variados.

A base do sistema está na classe `ProceduralRoad3D`, que define a estrada com base em âncoras (`anchors`) posicionadas ao longo de um percurso. A direção entre cada âncora é calculada com *Perlin Noise*, para garantir que as curvas sejam suaves e naturais. O parâmetro `smooth` controla a variação do ruído, enquanto o método `distanceBetweenAnchors()` determina o espaçamento entre os segmentos. Para adaptar a estrada ao terreno, a altura de cada âncora é ajustada com o `Terrain.activeTerrain.SampleHeight`, para assim garantir que a pista acompanhe o relevo do terreno.

Já a classe `RoadMesh` é responsável por converter o caminho definido pelas âncoras em uma malha 3D.

O processo envolve:

1. Definição dos vértices: Posicionados à esquerda e à direita do caminho central (largura da estrada).
2. Geração de triângulos: Conectando os vértices para formar as superfícies (topo, laterais e base).
3. Aplicação de materiais: Um material distinto para o asfalto e outro para a parte inferior da estrada.

O veículo, controlado pela classe `SimpleCarController`, utiliza `WheelColliders` para simular a física, incluindo aceleração, direção e colisões com o ambiente. O carro responde a *inputs* do utilizador. Na seguinte figura, encontra-se o teste da `ProceduralRoad3D` (a versão `ProceduralRoad` é semelhante, onde a estrada é criada segundo os planos x e z, enquanto esta versão utiliza os 3 planos para criar a estrada).

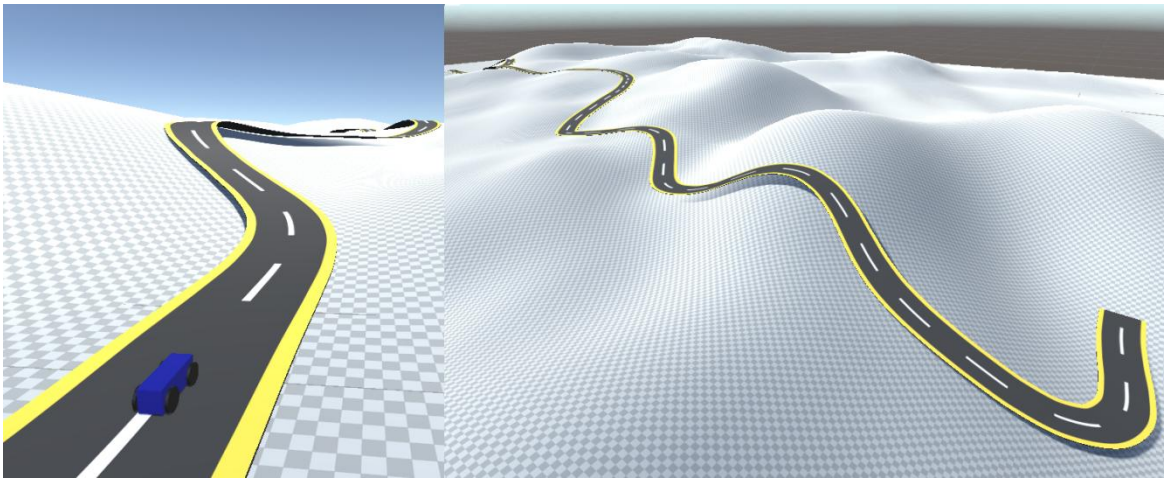


Figura 5 - Teste da ProceduralRoad3D

Ray Perception Sensor

Por fim desenvolvemos uma simulação em que o agente utiliza o componente `Ray Perception Sensors 3D` para detetar os componentes do ambiente e localizar uma esfera que deve alcançar. O agente dispara `raycasts` em várias direções num arco de 140º para identificar objetos ao seu redor (dispara `raycasts` entre -70º e +70º), como paredes e a própria esfera. Nesta tarefa, o agente começa num lado da plataforma, enquanto a esfera aparece no lado oposto. O objetivo é atravessar para o outro lado da plataforma, localizar a esfera e tocá-la, sem colidir

com as paredes, pois qualquer contacto termina o episódio com uma penalização.

O sistema de recompensas guia o comportamento do agente. Ao cruzar para o lado do campo onde a esfera está, o agente recebe uma recompensa positiva, de modo a encorajar essa transição. No entanto, se ele abandonar essa área sem ter alcançado a esfera, é penalizado com feedback negativo. O toque na esfera também resulta numa recompensa positiva, dado que é o objetivo final. Adicionalmente, tal como nas simulações anteriores, o contacto com paredes gera uma penalização imediata, e para evitar movimentos desnecessários, o agente apenas toma decisões a cada cinco passos, tendo assim escolhas mais deliberadas.

Esta simulação foi realizada inicialmente utilizando apenas o `Ray Perception Sensor 3D` para fornecer observações ao agente. Posteriormente, testou-se a adição de uma observação no método `CollectObservations()`:

```
public override void CollectObservations(VectorSensor sensor)
{
   //Indica se o agente está na sala do target
   sensor.AddObservation(targetRoom);
}
```

O objetivo era verificar se a adição desta informação iria melhorar o desempenho do treino. Contudo, os resultados mostraram que a adição desta observação não melhorou os resultados, mas sim piorou-os significativamente. Na seguinte tabela, encontram-se os resultados finais do treino dos agentes (resultados retirados do **TensorBoard**, gráfico das recompensas):

Configuração	Smoothed Reward	Raw Reward (Value)
Apenas `Ray Perception`	1.3585	1.3208
`Ray Perception` e `targetRoom`	-0.0224	0

Tabela 2 - Comparação entre configurações do agente

Este resultados indicam que o componente `Ray Perception Sensor 3D` foi suficiente para alcançar uma recompensa positiva, indicando assim que as observações deste componente já cobriam as necessidades de navegação do agente. A adição da observação no método `CollectObservations()` diminui o desempenho, o que indica que esta observação não era relevante para a realização das tarefas dos agentes ou que esta introduziu ruído na rede, confundindo assim o mesmo.

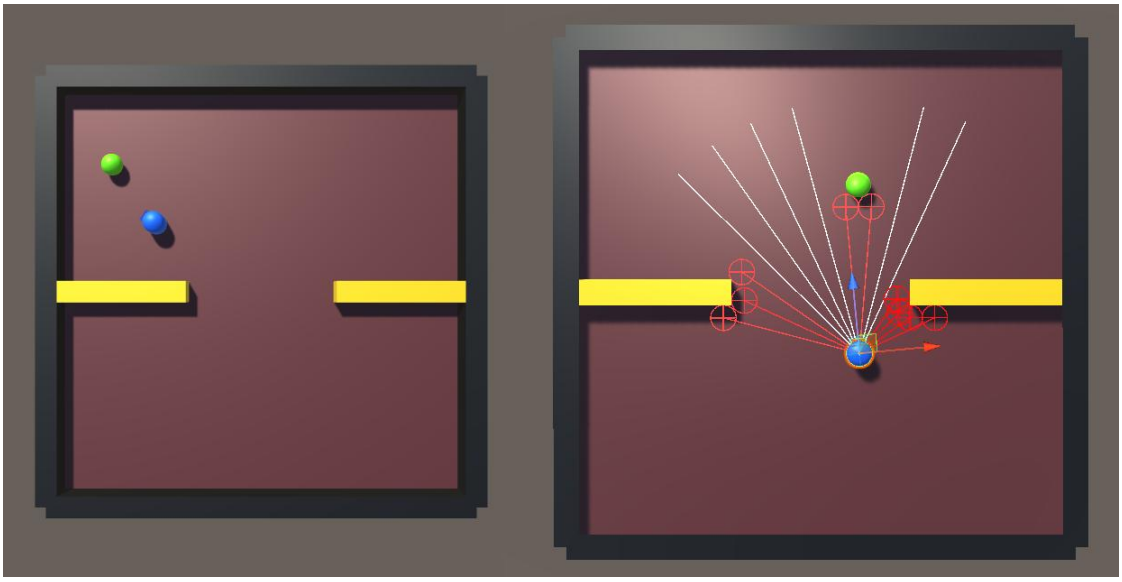


Figura 6 - Simulação RayPerceptionSensor

Na figura 6, encontra-se a simulação do agente (capsula azul), do lado esquerdo encontra-se uma simulação na janela do *Game* e do lado direito encontra-se uma simulação na janela do *Scene* demonstrando os `raycasts` do componente `Ray Perception Sensor 3D`, as linhas brancas indicam raios que não colidiram com nenhum objeto (atingiu o alcance máximo), as linhas a cor vermelho indicam raios que colidiram com um objeto (quanto mais próximo o objeto estiver, maior será a intensidade do vermelho).

Manager Scenes

Considerando todas as simulações realizadas anteriormente, o grupo decidiu desenvolver uma interface simples para facilitar o teste das diferentes cenas criadas. Para tal criou-se o script `ManagerScenes` para gerenciar a navegação entre cenas.



Figura 7 - Interface para escolha de uma cena

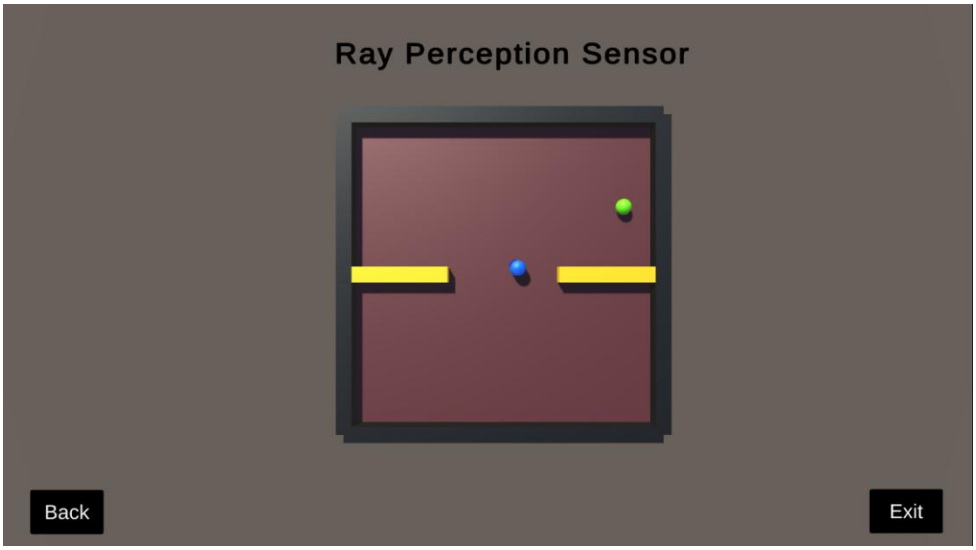


Figura 8 - Simulação RayPerceptionSensor apartir do ManagerScenes

Conclusão

O desenvolvimento deste projeto permitiu-nos explorar e aplicar diferentes técnicas de aprendizagem por reforço em ambientes criados no Unity. Através da construção de várias simulações com graus de complexidade diferentes, fomos capazes de testar e comparar métodos como aprendizagem convencional, aprendizagem por imitação e o treino com múltiplos ambientes em paralelo, sempre com o objetivo de melhorar o desempenho dos agentes. Estas simulações reforçaram a importância das recompensas e da perceção sensorial na eficácia do treino dos agentes. Para trabalhos futuros, seria interessante aprofundar a integração de sensores mais avançados e a criação de agentes para ambientes de jogo em forma de NPCs (*Non Playable Characters*).

Bibliografia

Consulta:

Vídeos gravados pelo Prof. Arnaldo Abrantes - Moodle

Wikipedia Contributors. (2019, April 29). Machine learning. Wikipedia; Wikimedia Foundation. https://en.wikipedia.org/wiki/Machine_learning

dos, C. (2005, April 28). Curva de Bézier. Wikipedia.org; Fundação Wikimedia, Inc. https://pt.wikipedia.org/wiki/Curva_de_B%C3%A9zier

Unity ML-Agents Toolkit. (n.d.). Unity-Technologies.github.io. https://unity-technologies.github.io/ml-agents/